

JavaScript Fundamentals — Solutions

Complete Corrections Guide · Arrays · Loops · Functions

● 3 Assignments

● Full Working Solutions

● Expected Outputs Included

Assignment 1 — Grade Tracker: Full Solution

01 / ARRAYS

Arrays · `.push()` · `.pop()` · Destructuring · Spread Operator

■ Marking Guide

All 6 tasks use the same two arrays. Check that students reuse the same array across tasks rather than re-declaring it each time. Common error: re-creating the arrays after `.pop()` — the array should reflect the mutation.

Task 1 Create an array called 'students' holding 5 student names as strings

Task 2 Create a second array called 'scores' holding their matching scores as numbers

✓ **SOLUTION**

CODE

```
// Tasks 1 & 2 – Declare the two core arrays
const students = ["Amara", "Tunde", "Bisi", "Chidi", "Ngozi"];
const scores   = [78, 85, 90, 62, 95];
```

Task 3 Use `.push()` to add a new student and score to both arrays

✓ **SOLUTION**

CODE

```
// Task 3 – .push() adds to the END of an array
students.push("Emeka");
scores.push(88);
// students → ["Amara", "Tunde", "Bisi", "Chidi", "Ngozi", "Emeka"]
// scores   → [78, 85, 90, 62, 95, 88]
```

■ EXPECTED CONSOLE OUTPUT

```
console.log(students.length) // → 6
console.log(scores[5])      // → 88
```

Task 4 Use `.pop()` to remove the last student and log what was removed

✓ **SOLUTION**

CODE

```
// Task 4 – .pop() removes & RETURNS the last element
const removed = students.pop(); // .pop() returns the removed value
scores.pop(); // also remove the matching score
console.log("Removed:", removed);
```

EXPECTED CONSOLE OUTPUT

Removed: Emeka

Task 5 Use destructuring to pull the first two students into variables

✓ **SOLUTION**

CODE

```
// Task 5 – Array destructuring unpacks by position
const [first, second] = students;
console.log(first, second);
```

EXPECTED CONSOLE OUTPUT

Amara Tunde

Task 6 Use the spread operator to combine both arrays into classData

✓ **SOLUTION**

CODE

```
// Task 6 – Spread (...) expands array elements in-place
const classData = [...students, ...scores];
console.log(classData);
```

EXPECTED CONSOLE OUTPUT

["Amara", "Tunde", "Bisi", "Chidi", "Ngozi", 78, 85, 90, 62, 95]

■ **BONUS SOLUTION**

BONUS

```
// Bonus – .indexOf() and .includes()
const pos = students.indexOf("Bisi");
console.log(pos); // → 2 (zero-indexed)

const has100 = scores.includes(100);
console.log(has100); // → false
```

■ EXPECTED CONSOLE OUTPUT

```
pos    → 2  
has100 → false
```

Assignment 2 — FizzBuzz & Class Report: Full Solution

02 / LOOPS

For Loops · While Loops · if/else · Modulo Operator

■ Marking Guide

For Part A, all three tasks (loop output, grade letters, running average) should appear INSIDE or AFTER one single loop — not three separate loops. Common errors: (1) using == instead of === for comparisons; (2) forgetting i++ in the while loop (infinite loop); (3) checking i%3 before i%15 in FizzBuzz — the FizzBuzz check must come first.

PART A — CLASS REPORT CARD

Task 1 For loop printing "Name scored Score" for each student

Task 2 Add if/else inside the loop for grade letters: A/B/C/F

Task 3 Running total and class average calculated after the loop

✓ SOLUTION

CODE

```
// Tasks 1, 2, 3 — One loop handles all three tasks
const students = ["Amara", "Tunde", "Bisi", "Chidi", "Ngozi"];
const scores = [78, 85, 90, 62, 95];
let total = 0;

for (let i = 0; i < students.length; i++) {
  console.log(`${students[i]} scored ${scores[i]}`);

  let grade;
  if (scores[i] >= 90) grade = "A";
  else if (scores[i] >= 75) grade = "B";
  else if (scores[i] >= 60) grade = "C";
  else grade = "F";
  console.log(`Grade: ${grade}`);

  total += scores[i]; // running total — Task 3
}

const average = total / students.length;
console.log("Class average:", average);
```

■ EXPECTED CONSOLE OUTPUT

```
Amara scored 78    → Grade: B
Tunde scored 85    → Grade: B
Bisi scored 90     → Grade: A
Chidi scored 62    → Grade: C
Ngozi scored 95    → Grade: A
Class average: 82
```

PART B — FIZZBUZZ

Task 4 FizzBuzz 1–30 using a for loop

Task 5 Re-write the same FizzBuzz using a while loop

✓ SOLUTION

CODE

```
// Task 4 – FizzBuzz with for loop
// KEY: check i%15 FIRST – it must come before i%3 and i%5
for (let i = 1; i <= 30; i++) {
  if (i % 15 === 0) console.log("FizzBuzz");
  else if (i % 3 === 0) console.log("Fizz");
  else if (i % 5 === 0) console.log("Buzz");
  else console.log(i);
}

// Task 5 – Exact same logic, while loop instead
let i = 1;
while (i <= 30) {
  if (i % 15 === 0) console.log("FizzBuzz");
  else if (i % 3 === 0) console.log("Fizz");
  else if (i % 5 === 0) console.log("Buzz");
  else console.log(i);
  i++; // ■■ MUST increment – or infinite loop!
}
```

■ EXPECTED CONSOLE OUTPUT

```
1, 2, Fizz, 4, Buzz, Fizz, 7, 8, Fizz, Buzz,
11, Fizz, 13, 14, FizzBuzz, 16, 17, Fizz, 19, Buzz,
Fizz, 22, 23, Fizz, Buzz, 26, Fizz, 28, 29, FizzBuzz
```

■■ Common Mistake — FizzBuzz Order

Students often check `i%3` before `i%15`. If 15 is divisible by both 3 and 5, the `i%3` branch fires first and prints "Fizz" instead of "FizzBuzz". The combined check (`i%15 or i%3===0 && i%5===0`) must always come first.

■ BONUS SOLUTION

BONUS

```
// Bonus – for...of with scores.entries() to access both index and value
const average = total / students.length; // from Part A

for (const [i, score] of scores.entries()) {
  if (score > average) {
    console.log(`${students[i]} scored above average: ${score}`);
  }
}
```

■ EXPECTED CONSOLE OUTPUT

```
Tunde → 85 (above 82)
Bisi → 90 (above 82)
Ngozi → 95 (above 82)
```

Assignment 3 — Functions & Higher-Order Magic: Full Solution

03 / FUNCTIONS

Functions · Arrow Functions · `.map()` · `.filter()` · `.reduce()`

■ Marking Guide

Part A: The regular function and arrow function must produce identical results. Part B: For Task 5, students need to filter by `scores[index]` — not directly on the `students` array. For Task 7, the chained `.filter().map()` must still reference the correct index. Common error: Task 6 — forgetting the initial value (0) in `.reduce()` — causes NaN.

PART A — REUSABLE FUNCTIONS

- Task 1** Write `getGrade(score)` — regular function returning A, B, C, or F
- Task 2** Write `getGradeArrow` — arrow function version of the same logic
- Task 3** Write `announce(name, score)` returning "Amara — Grade: B (85)"

✓ SOLUTION

```
CODE

// Task 1 — Regular function declaration
function getGrade(score) {
  if (score >= 90) return "A";
  if (score >= 75) return "B";
  if (score >= 60) return "C";
  return "F";
}

// Task 2 — Arrow function (identical behaviour, modern syntax)
const getGradeArrow = (score) => {
  if (score >= 90) return "A";
  if (score >= 75) return "B";
  if (score >= 60) return "C";
  return "F";
};

// Task 3 — Template literal assembles the output string
function announce(name, score) {
  return `${name} — Grade: ${getGrade(score)} (${score})`;
}
console.log(announce("Amara", 78));
```

■ EXPECTED CONSOLE OUTPUT

```
Amara — Grade: B (78)
```

PART B — MAP, FILTER & REDUCE

- Task 4** Use `.map()` with `getGrade` to create a grades array of letter grades
- Task 5** Use `.filter()` to get only the students who passed (`score ≥ 60`)
- Task 6** Use `.reduce()` to calculate the total of all scores (no for loop!)

Task 7 Chain `.filter()` + `.map()` to get only the names of A-grade students

✓ SOLUTION

CODE

```
// Task 4 – .map() transforms every element via a function
const grades = scores.map(score => getGrade(score));
// Shorthand: scores.map(getGrade) ← same result

// Task 5 – .filter() keeps elements where callback returns true
// We need the index to look up the matching score
const passed = students.filter((_, i) => scores[i] >= 60);

// Task 6 – .reduce() accumulates a single value
// ■■ The second arg (0) is the initial value – never omit it!
const totalScore = scores.reduce((sum, s) => sum + s, 0);

// Task 7 – Chain: filter first, then map the remaining names
const aStudents = students
  .filter((_, i) => scores[i] >= 90)
  .map(name => name); // .map(name=>name) keeps name as-is

console.log(grades, passed, totalScore, aStudents);
```

■ EXPECTED CONSOLE OUTPUT

```
grades      → ["B","B","A","C","A"]
passed      → ["Amara","Tunde","Bisi","Chidi","Ngozi"]
totalScore  → 410
aStudents   → ["Bisi","Ngozi"]
```

■■ Common Mistake — `.reduce()` without initial value

`scores.reduce((sum, s) => sum + s)` — missing the 0 — still works here because `scores` is never empty, but it FAILS on an empty array. Always pass the initial value as the second argument.

■ BONUS SOLUTION

BONUS

```
// Bonus – one chain: filter passing → reduce to average
// The trick: divide each element by arr.length inside the callback
const passingAvg = scores
  .filter(s => s >= 60)
  .reduce((sum, s, _, arr) => sum + s / arr.length, 0);

console.log(passingAvg);
```

■ EXPECTED CONSOLE OUTPUT

```
passingAvg → 82
```